

2.create a Local repository, clone repository using GitHub and Explain about Jenkins :

What is GitHub :

GitHub is a web-based platform that uses Git for version control. It allows developers to store and manage their code, track changes, collaborate with others, and work on multiple projects simultaneously. GitHub is popular for open-source projects, team collaboration, and code sharing.

Key Concepts:

Git: A distributed version control system that tracks changes in source code during software development.

Repository: A project or folder that contains all files and history related to the project. In GitHub, a repository can be hosted online (remote repository).

Local Repository :-

A local repository is a Git repository that exists on your local machine. It's where you can make changes, commit them, and manage your code without affecting the main (remote) repository until you push those changes.

Example :- Suppose you have a project on your local computer. You initialize it as a Git repository using the following command : **git init**.

Now, this folder is a local Git repository where you can track changes.

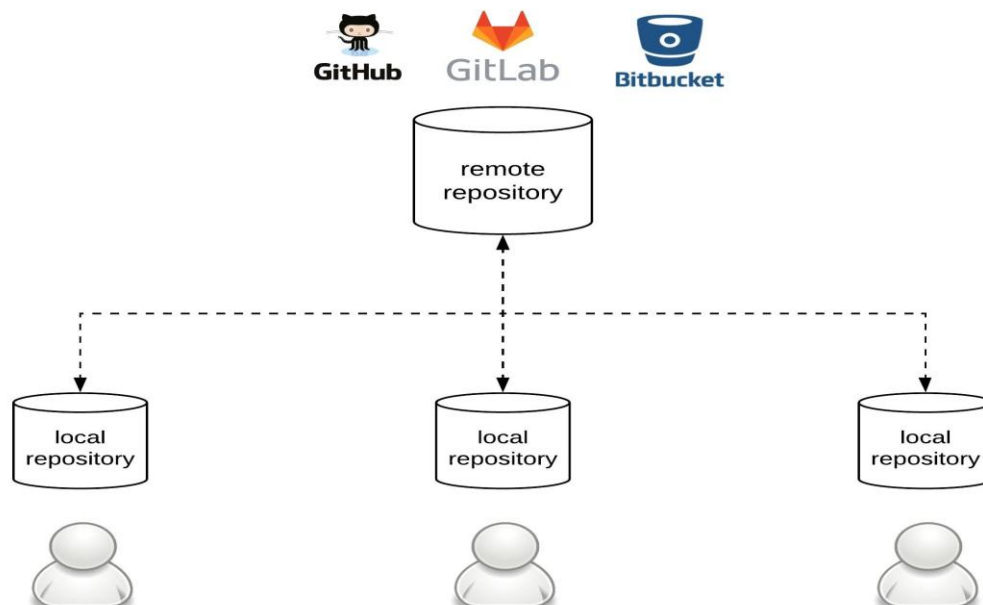


Fig : GitHub Local Repository

Clone Repository :-

Cloning a repository means creating a copy of a remote GitHub repository on your local machine. This allows you to work on the project locally, make changes, and then push those changes back to the remote repository on GitHub.

Example :- To clone a repository from GitHub,

you would use: ***git clone*** <https://github.com/username/repository-name.git>

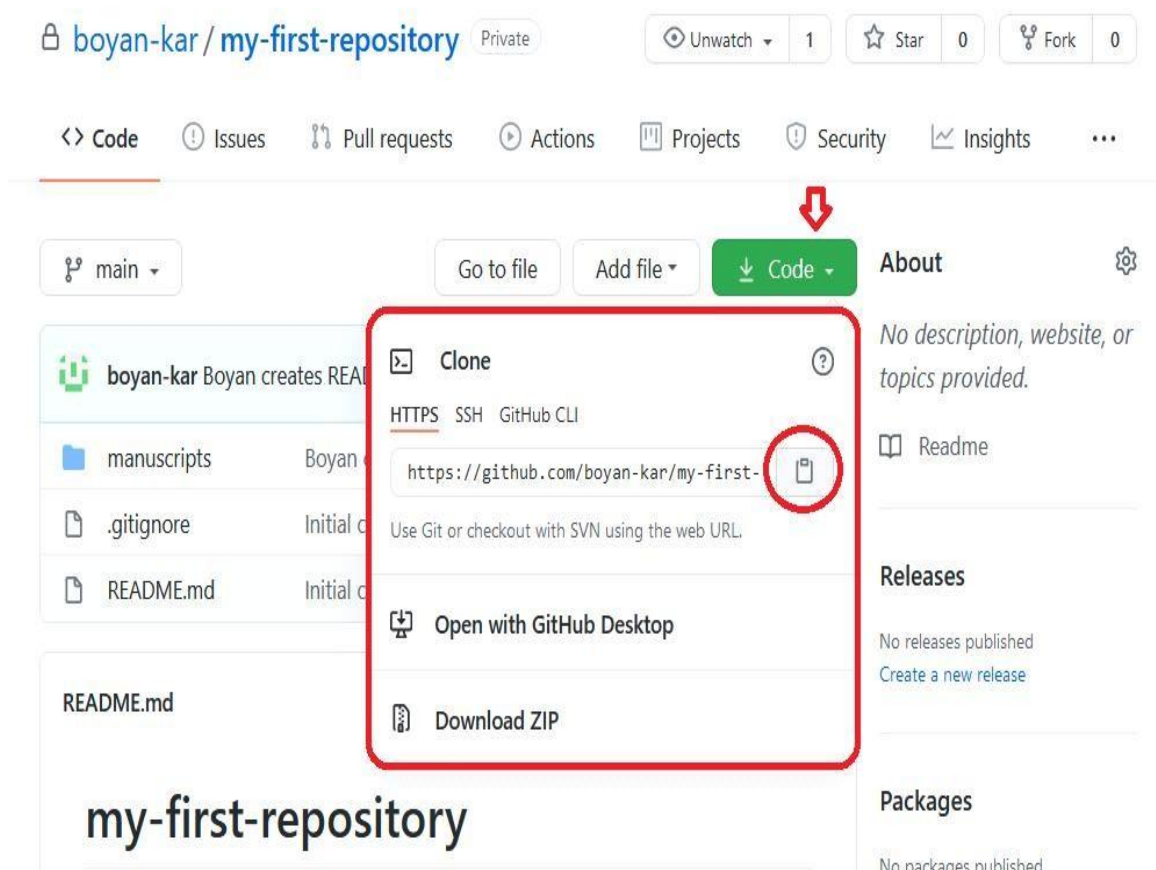


Fig : GitHub Clone Repository

What is Jenkins :-

Jenkins is an open-source automation server used to automate parts of the software development process, such as building, testing, and deploying applications. It's widely used for continuous integration (CI) and continuous delivery (CD) in software projects.

Key Features of Jenkins:

Automation: Jenkins automates repetitive tasks, like code integration, builds, and testing, making the development process more efficient.

Plugins: Jenkins has a vast ecosystem of plugins that extend its functionality. These plugins allow Jenkins to integrate with many other tools and technologies, such as GitHub, Docker, Maven, etc.

Continuous Integration (CI): Jenkins helps developers integrate their code changes into a shared repository frequently, automatically building and testing the code to ensure it works correctly.

Continuous Delivery (CD): After testing, Jenkins can automatically deploy the code to production or a staging environment, making the software delivery process faster and more reliable.

Pipeline as Code: Jenkins supports defining CI/CD pipelines as code using a Domain-Specific Language (DSL). This allows teams to version control their build pipelines along with their codebase.

Example Workflow with Jenkins:

Commit Code: A developer commits code to a version control system like GitHub.

Automated Build: Jenkins detects the commit and triggers an automated build of the code.

Testing: Jenkins runs automated tests to validate the code changes.

Deployment: If tests pass, Jenkins can automatically deploy the application to a staging or production environment.

Software / Tools Setup for React Experiments (Exp 3–5)

Software Required :-

1. Node.js (LTS Version)
2. npm (comes with Node.js)
3. Visual Studio Code

Installation Steps:-

Step 1 – Install Node.js

Download from <https://nodejs.org>

Verify installation in terminal/command prompt: **node -v**

Step 2 – Install Visual Studio Code

1. Download from <https://code.visualstudio.com>.
2. Install extensions (optional): *React Developer Tools, Prettier*.

Step 3 – Create a New React Project

Run the following commands in Command Prompt/Terminal:

1. **npx create-react-app my-app**
 - **npx** → Runs a package without installing it globally.
 - **create-react-app** → Generates a new React project with default setup.
 - **my-app** → Name of the project folder.
2. **cd my-app**
 - **cd** means Change Directory.
 - This command moves into the new **my-app** folder that was just created.
3. **npm start**
 - **npm** is Node Package Manager.
 - **start** runs the React development server.
 - The application will open in the browser at **http://localhost:3000**

Step 4 – Add Your Experiment Code

- Open the project in VS Code.
- Replace the contents of **src/App.js** and **src/App.css** with your experiment code.
- Save the files → Browser will automatically refresh with the new changes.

3. Create a code using HTML, React, JS (Registration Page, Login Page, Feedback Page):-

App.js

```
import React, { useState } from 'react';
import './App.css';
function RegistrationPage({ onSwitch }) {
  const Register = () => {
    alert('Registration Successful!');
    onSwitch('login');
  };
  return (
    <div className="form-container">
      <h2>Registration</h2>
      <form onSubmit={Register}>
        <input type="text" placeholder="First Name" required /><br/>
        <input type="text" placeholder="Last Name" required /><br />
        <input type="email" placeholder="Email" required /><br />
        <input type="password" placeholder="Password" required /><br />
        <button type="submit">Register</button><br />
      </form>
    </div>
  );
}
function LoginPage({ onSwitch }) {
  const Login = () => {
    alert('Login Successful!');
    onSwitch('feedback');
  };
  return (
    <div className="form-container">
      <h2>Login</h2>
      <form onSubmit={Login}>
```

```
    <input type="email" placeholder="Email" required /><br />
    <input type="password" placeholder="Password" required /><br />
    <button type="submit">Login</button><br />
  </form>
</div>
);
}
function FeedbackForm({ onSwitch }) {
  const Feedback = () =>{
    alert('Feedback submitted. Thank you!');
    onSwitch('thankYou');
  };
  return (
    <div className="form-container">
      <h2>Feedback Form</h2>
      <form onSubmit={Feedback}>
        <input type="text" placeholder="Your Name" required /><br />
        <input type="email" placeholder="Your Email" required /><br />
        <textarea placeholder="Your Feedback" required></textarea><br />
        <button type="submit">Submit</button><br />
      </form>
    </div>
  );
}
function ThankYouPage({ onSwitch }) {
  return (
    <div className="form-container">
      <h2>Thank You!</h2>
      <p>Your feedback has been received.</p>
    </div>
  );
}
```

```
function App() {  
  const [page, setPage] = useState('register');  
  return (  
    <div className="App">  
      {page === 'register' && <RegistrationPage onSwitch={setPage} />}  
      {page === 'login' && <LoginPage onSwitch={setPage} />}  
      {page === 'feedback' && <FeedbackForm onSwitch={setPage} />}  
      {page === 'thankYou' && <ThankYouPage onSwitch={setPage} />}  
    </div>  
  );  
}  
export default App;
```

App.css

```
.form-container {  
  width: 300px;  
  margin: 50px auto;  
  padding: 20px;  
  border: 1px solid #ccc;  
  border-radius: 8px;  
  text-align: center;  
  color:darkblue;  
}  
input, textarea, button {  
  width: 100%;  
  margin: 8px 0;  
  padding: 10px;  
  box-sizing: border-box;  
}  
.link {
```

```
color: blue;
cursor: pointer;
text-decoration: underline;
}
```

4.create a code using React Static and Dynamic, and create a code using React counter (Hook concept):

1. Create a React app :

```
npx create-react-app react-demo
cd react-demo
npm install react-router-dom
```

2. Inside src/component/, create 4 files:

- login.js (Login Page)
- home.js (Home Page)
- about.js (About Page)
- contact.js (Contact Page)

3. Code:

App.js

```
import React from 'react';
import { BrowserRouter as Router, Route, Routes, Link } from 'react-router-dom';
import Login from './component/login';
import Home from './component/home';
import AboutUs from './component/about';
import Contact from './component/contact';
import './App.css';
function App() {
  return (
    <center>
      <Router>
        <div className="App">
```

```
<nav className="App-nav">
  <ul>
    <li><Link to="/login">Login</Link></li>
    <li><Link to="/">Home</Link></li>
    <li><Link to="/about">About Us</Link></li>
    <li><Link to="/contact">Contact</Link></li>
  </ul>
</nav>

<Routes>
  <Route path="/login" element={<Login />} />
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<AboutUs />} />
  <Route path="/contact" element={<Contact />} />
</Routes>

</div>

</Router>

</center>

);
}

export default App;
```

App.css

```
.App-nav {
  background: #333;
  color: #fff;
  padding: 1rem;
}

.App-nav ul {
  list-style: none;
  padding: 0;
  margin: 0;
```

```
display: flex;
}
.App-nav li {
margin-right: 1rem; }
.App-nav a {
color: #fff;
text-decoration: none; }
.App-nav a:hover {
text-decoration: underline; }
```

Login.js

```
import React from "react";
import { useNavigate } from "react-router-dom";
function Login() {
  const navigate = useNavigate();
  const handleSubmit = (e) => {
    e.preventDefault(); // prevent page reload
    navigate("/"); // go to Home page
  };
  return (
    <form onSubmit={handleSubmit}>
      <center>
        <h2>Login</h2>
        <input type="text" placeholder="Name" /><br />
        <input type="password" placeholder="Password" /><br />
        <button type="submit" style={{ backgroundColor: 'tomato', color:'white'}}>
          Submit
        </button>
      </center>
    </form>
  );
}
```

```
}  
  
export default Login;
```

Home.js

```
import React from "react";  
  
function Home() {  
  return (  
    <div style={{ backgroundColor: 'lightgrey'}}>  
      <center>  
        <h2>Home</h2>  
        <p>Welcome to VSNS website</p>  
      </center>  
    </div>  
  );  
}  
  
export default Home;
```

About

```
import React from "react";  
  
function AboutUs() {  
  return (  
    <center><div style={{ backgroundColor: 'lightgrey'}}>  
      <h2>AboutUs</h2>  
      <p>about us</p>  
    </div></center>  
  );  
}  
  
export default AboutUs;
```

Contact

```
import React from "react";  
  
function Contact() {  
  return (  
    <div style={{ backgroundColor: 'lightgrey'}}><center>  
      <h2>Contact</h2>
```

```
        <p>contact details</p>
      </center></div>

    );
  }

  export default Contact;
```

Counter (without using Hook):

App.js

```
import React from 'react';

function App() {
  let count = 15;
  const addValue = () => {
    count += 1;
    console.log("clicked", count);
  };
  return (
    <div>
      <h1 id="count-display">Count: {count}</h1>
      <button onClick={addValue}>Increase Count</button>
    </div>
  );
}

export default App;
```

App.css

```
.App {
  text-align: center;
}

.App-logo {
  height: 40vmin;
  pointer-events: none;
```

```
}
@media (prefers-reduced-motion: no-preference) {
  .App-logo {
    animation: App-logo-spin infinite 20s linear;
  }
}
.App-header {
  background-color: #282c34;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  font-size: calc(10px + 2vmin);
  color: white;
}
.App-link {
  color: #61dafb;
}
@keyframes App-logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}
```

Counter (with Using Hook)

App.js

```
import React, { useState } from 'react';
```

```
function Counter() {  
  const [count, setCount] = useState(0);  
  const increaseCount = () => {  
    setCount(count + 1); // triggers re-render  
  };  
  return (  
    <div>  
      <h1>Count: {count}</h1>  
      <button onClick={increaseCount}>Increase</button>  
    </div>  
  );  
}  
export default Counter;
```

App.css

```
.App {  
  text-align: center;  
}  
.App-logo {  
  height: 40vmin;  
  pointer-events: none;  
}  
@media (prefers-reduced-motion: no-preference) {  
  .App-logo {  
    animation: App-logo-spin infinite 20s linear;  
  }  
}  
  
.App-header {  
  background-color: #282c34;  
  min-height: 100vh;
```

```
display: flex;
flex-direction: column;
align-items: center;
justify-content: center;
font-size: calc(10px + 2vmin);
color: white;
}
.App-link {
  color: #61dafb;
}
@keyframes App-logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}
```

5.Create code using java script a Shopping Cart.

App.js

```
import './App.css';
import { useState, useEffect } from 'react';
//header function
const Header = ({ count, handleShow }) => (
  <div className="flex shopping-card">
    <div onClick={() => handleShow(false)}>ShoppingCart App</div>
    <div onClick={() => handleShow(true)}>Cart <sup>{count}</sup></div>
  </div>
);
//productlist function
```

```
const ProductList = ({ product, addToCart }) => (
  <div className="flex product-list">
    {product.map((item, i) => (
      <div key={i} className="product-item">
        <img src={item.url} alt={item.name} width="100%" />
        <p>{item.name} | {item.category}</p>
        <p>{item.seller}</p>
        <p>Rs. {item.price}</p>
        <button onClick={() => addToCart(item)}>Add To Cart</button>
      </div>
    ))}
  </div>
);

//cartlist function
const CartList = ({ cart }) => {
  const [CART, setCART] = useState([]);
  useEffect(() => setCART(cart), [cart]);
  const updateQuantity = (i, delta) => {
    const updated = CART.map((item, idx) =>
      idx === i ? { ...item, quantity: Math.max(1, item.quantity + delta) } : item
    );
    setCART(updated);
  };
  const total = CART.reduce((sum, item) => sum + item.price * item.quantity, 0);
  return (
    <div className="cart-list">
      {CART.map((item, i) => (
        <div key={i} className="cart-item">
          <img src={item.url} alt={item.name} width={40} />
          <span>{item.name}</span>
        </div>
      ))}
    </div>
  );
};
```

```

        <button onClick={() => updateQuantity(i, -1)}>-</button>

        <span>{item.quantity}</span>

        <button onClick={() => updateQuantity(i, 1)}>+</button>

        <span>Rs. {item.price * item.quantity}</span>

    </div>

    )))}

    <p><strong>Total:</strong> Rs. {total}</p>

</div>

);

};

// app function
export default function App() {
    const [products] = useState([
        {
            url: "https://m.media-amazon.com/images/I/71nvkHnPPZL._SX679_.jpg",
            name: "IPHONE 16PRO MAX",
            category: "IPHONE",
            seller: "APPLE",
            price: 70000,
        },
    ]);

    const [cart, setCart] = useState([]);
    const [showCart, setShowCart] = useState(false);
    const addToCart = item => setCart([...cart, { ...item, quantity: 1 }]);

    return (
        <div>

            <Header count={cart.length} handleShow={setShowCart} />

            {showCart ? <CartList cart={cart} /> : <ProductList product={products}
            addToCart={addToCart} />}

        </div>

    );
}

```

```
}
```

App.css

```
.flex, .shopping-card, .cart-item { display: flex; }  
.flex { flex-wrap: wrap; }  
.shopping-card {  
  justify-content: space-between;  
  align-items: center;  
  background: #61dafb;  
  padding: 20px 30px;  
  font: bold 20px sans-serif;  
  cursor: pointer;  
}  
.product-item, .cart-item {  
  padding: 20px; margin: 10px; text-align: center;  
  border-radius: 10px; border: 1px solid #ddd;  
  box-shadow: 0 2px 6px rgba(0,0,0,0.1); gap: 10px;  
  align-items: center;  
}  
.product-item { width: 30%; }  
.product-item img {  
  max-height: 400px;  
  margin-bottom: 10px;  
}  
.product-item button, .cart-item button {  
  padding: 6px 12px; font-size: 14px;  
  border-radius: 4px; cursor: pointer;  
}
```

6. MySql Database connectivity (Eclipse):

1. Install and Setup MySQL

- Install MySQL Server (MySQL Installer).

-
- Create a database:

CREATE DATABASE College;

- Create a table (example):

**CREATE TABLE Student (
name VARCHAR(50),
roll VARCHAR(10)
);**

- Insert sample data if needed:

INSERT INTO student VALUES ('robot', '12');

2. Install JDBC Driver

- Download MySQL JDBC Connector (mysql-connector-java-x.x.x.jar) from official MySQL website.
- Extract it somewhere on your PC.

3. Setup Eclipse Project

- Open **Eclipse IDE** → Create a **Java Project**.
- Inside src, create a new package (e.g., vijay).
- Create a new class (e.g., Connector.java).

Connector.java

```
package vijay;  
  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.SQLException;  
import java.sql.Statement;  
  
public class Connector {  
    public static void main(String[] args) throws SQLException {  
        Connection con = null;  
        Statement stat = null;  
        con = DriverManager.getConnection("jdbc:mysql://localhost:3306/College","root","1234");  
        stat = con.createStatement();  
        System.out.println("Connected to Database");  
    }  
}
```

```
String SQL = "INSERT INTO student (name, roll) VALUES ('vijay','13')";
// String SQL = "UPDATE student SET name='kumar' WHERE roll='12'";
// String SQL = "DELETE FROM student WHERE roll='13'";

stat.execute(SQL);

stat.close();

con.close();

System.out.println("Disconnected from Database, all operations done");
}
}
```

4. Add JDBC Driver to Project

- Right-click the project → Properties → Java Build Path.
- Go to Libraries → click Add External JARs....
- Select the mysql-connector-java-x.x.x.jar file → Apply & Close.

Common Errors & Fixes (Connector.java)

- **If you get errors while running your JDBC program, check the following:**

1. Fix module-info.java:

- If your project contains module-info.java, add:

```
module data {
requires java.sql;
}
```

7a. Creating a REST API with Spring Security (Spring Boot)

This experiment demonstrates how to create a secure REST API using Spring Boot with Spring Security, H2 Database, and Spring Data JPA.

Step 1: Generate a Spring Boot Project

1. Open your browser and go to <https://start.spring.io> (Spring Initializr).
2. Configure the project as follows:
 - Project: Maven
 - Language: Java
 - Spring Boot Version: Latest stable version
 - Packaging: Jar

-
- Java: 17
3. Add the following dependencies:
 - Spring Web
 - Spring Security
 - Spring Data JPA
 - H2 Database
 4. Click Generate to download the project as a .zip file.
 5. Extract the .zip file to a suitable location.

Step 2: Import Project into Eclipse

1. Open Eclipse IDE.
2. Go to File → Import → Existing Maven Project.
3. Browse and select the extracted project folder.
4. Click Finish.
5. Wait until Maven finishes downloading the required dependencies.

Step 3: Create Java Classes

Inside `src/main/java/com/example/rest/`, create the following files:

(a) Default Main Application (Auto-generated)

DemoApplication.java

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;

@SpringBootApplication
@EnableWebSecurity

public class demoApplication {

    public static void main(String[] args) {

        SpringApplication.run(demoApplication.class, args);

    }
}
```

(b) REST Controller

MessageController.java

```
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController

public class Messagecontroller {

    @GetMapping("/")

    public String getMessage() {

        return "<h1>welcome to spring boot security</h1>";

    }
}
```

Step 3: Configure Application Properties

- Open the file: src/main/resources/application.properties
and add the following configuration:

```
spring.security.user.name=vijay
spring.security.user.password=123
```

When prompted for login, enter:

- **Username:** vijay **Password:** 123

7b. How to Create a RESTful Service (Spring Boot)

This experiment demonstrates how to create a simple RESTful web service using Spring Boot.

Step 1: Install Spring Tool Suite (STS)

1. Download Spring Tool Suite (STS) from: <https://spring.io/tools>
2. Extract the downloaded file.
3. Launch STS by running the executable file.

Step 2: Create a New Spring Starter Project

1. In STS, go to File → New → Spring Starter Project.
2. If not listed, click on Other..., type Spring Starter Project, select it, then click Next.

Step 3: Provide Project Information

Fill in the project details as follows:

- **Name:** restful-web-services

-
- **Group:** com.javatpoint
 - **Package:** com.javatpoint.server.main
 - Click **Next**.

Step 4: Add Dependencies

Choose **Spring Boot Version 2.1.8** and add the following dependencies:

- Spring Web (Spring MVC)
- Spring Boot DevTools
- Spring Data JPA
- H2 Database

Step 5: We can see the project structure in the project explorer window.

Step 6: Create a REST Controller

Create a new class in the package com.javatpoint.server.main:

HelloWorldController.java

```
package com.javapoint.server.nain;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

//controller

@RestController

public class HelloWorldController
{
    @GetMapping(path="/hello-world")
    public String helloWorld() {
        return "Hello World";}

    @GetMapping(path="/hello-world-bean")
    public HelloWorldBean helloWorldBean() {
        return new HelloWorldBean("Hello World");
    }
}
```

Step 7: Create a Bean Class

Create another class in the same package:

HelloWorldBean.java

```
package com.javapoint.server.nain;

public class HelloWorldBean {

    public String message;

    public HelloWorldBean(String message) {

        this.message=message;

    }

    //generating getter and setter

    public String getMessage() {

        return message;

    }

    public void setMessage(String message)

    {

        this.message=message;

    }

    @Override

    public String toString() {

        return String.format("HelloWorldBean [message=%s]",message);

    }

}}
```

OUTPUT

<http://localhost:8080/hello-world>

<http://localhost:8080/hello-world-bean>

8. Write the code for Constructor and Setter injection (in Spring)

A. Constructor Injection (using Eclipse)

Step 1: Open Eclipse IDE → Create a Java Project (e.g., SpringConstructorInjection).

Step 2: Inside src/, create a package com.example.

Step 3: Create the following files:

Engine.java

```
package com.example;

public class Engine {

    public void start() {
```

```
        System.out.println("Engine started!");
    }
}
```

Car.java

```
package com.example;

public class Car {
    private Engine engine;
    // Constructor Injection
    public Car(Engine engine) {
        this.engine = engine;
    }
    public void drive() {
        engine.start();
        System.out.println("Car is driving!");
    }
}
```

Main.java

```
package com.example;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Main {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("spring-constructor-injection.xml");
        Car car = (Car) context.getBean("car");
        car.drive();
    }
}
```

spring-constructor-injection.xml (put inside src/)

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<!-- Define Engine bean -->
<bean id="engine" class="com.example.Engine" />
<!-- Define Car bean using constructor injection -->
<bean id="car" class="com.example.Car">
    <constructor-arg ref="engine" />
</bean>
</beans>
```

Step 4: Run Main.java.

Output:

Engine started!

Car is driving!

B. Setter Injection (using Eclipse)

Step 1: Create a new Java Project (e.g., SpringSetterInjection).

Step 2: Inside src/, create package com.example.

Step 3: Create the following files:

Engine.java

```
package com.example;

public class Engine {
    public void start() {
        System.out.println("Engine started!");
    }
}
```

Car.java

```
package com.example;
```

```
public class Car {  
    private Engine engine;  
    // Setter Injection  
    public void setEngine(Engine engine) {  
        this.engine = engine;  
    }  
    public void drive() {  
        engine.start();  
        System.out.println("Car is driving!");  
    }  
}
```

Main.java

```
package com.example;  
  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
  
public class Main {  
    public static void main(String[] args) {  
        ApplicationContext context =  
            new ClassPathXmlApplicationContext("spring-setter-injection.xml");  
        Car car = (Car) context.getBean("car");  
        car.drive();  
    }  
}
```

spring-setter-injection.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://www.springframework.org/schema/beans  
        http://www.springframework.org/schema/beans/spring-beans.xsd">  
  
    <!-- Define Engine bean -->  
  
    <bean id="engine" class="com.example.Engine" />
```

```
<!-- Define Car bean using setter injection -->
<bean id="car" class="com.example.Car">
    <property name="engine" ref="engine" />
</bean>
</beans>
```

Step 4: Run Main.java.

Output:

Engine started!

Car is driving!

C. Using Spring Initializr (Spring IO)

Step 1: Open <https://start.spring.io>.

- Project: Maven
- Language: Java
- Spring Boot Version: Latest stable
- Dependencies: Spring Context

Step 2: Generate and download the project.

Step 3: Import the project into Eclipse/IntelliJ.

Step 4: Add Engine and Car classes inside src/main/java.

Step 5: Run the project

9. Create REST controller for CRUD operations and test with Postman (MongoDB):

1. MongoDB Installation Process

1. Go to MongoDB Community Server Download.
2. Download the installer for your OS.
3. Run the installer → choose Complete Setup.
4. After installation, start the MongoDB server.

2. Postman Installation Process

-
1. Download Postman from <https://www.postman.com/downloads/>.
 2. Install and open it.
 3. You will use Postman to send API requests (GET, POST, PUT, DELETE).

3. Project, Folder and File Creation in Eclipse

1. Open Eclipse IDE → File → New → Project → Crud(name of the project)
2. Inside project create a folder (models).
3. Inside it, create:
 - index.js → main server file (Crud)
 - models/user.js → /model/
4. Initialize npm command in terminal:

```
npm init -y
```

```
npm install express mongoose
```

4.Code.

Index.js

```
const express = require("express");
```

```
const mongoose = require("mongoose");
```

```
const User = require("./models/user");
```

```
const app = express();
```

```
app.use(express.json());
```

```
// Connect DB
```

```
mongoose.connect("mongodb://localhost:27017/mongodb")
```

```
.then(() => console.log(" Connected to MongoDB"))
```

```
.catch(err => console.error(" DB Error:", err));
```

```
// Routes
```

```
app.post("/users", (req, res) => User.create(req.body).then(u => res.json(u)).catch(e => res.status(400).json(e)));
```

```
app.get("/users", (req, res) => User.find().then(u => res.json(u)).catch(e => res.status(500).json(e))
```

```
);
```

```
app.get("/users/:id", (req, res) => User.findById(req.params.id).then(u => u ? res.json(u) : res.status(404).send("Not found")) .catch(e => res.status(500).json(e)));
```

```
app.put("/users/:id", (req,res)=>User.findByIdAndUpdate(req.params.id, req.body,
{new:true}).then(u=>u?res.json(u):res.status(404).send("Notfound")).catch(e=>res.status(400).json(e)));
```

```
app.delete("/users/:id", (req,res)=>User.findByIdAndDelete(req.params.id).then(u=>u?res.json("Deleted"):res.status(404).send("Not found")).catch(e=>res.status(500).json(e)));
```

```
app.listen(3000, ()=>console.log(" Server running at http://localhost:3000"));
```

User.js

```
const mongoose = require("mongoose");
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  age: { type: Number, required: true },
  gender: { type: String, required: true }
});
module.exports = mongoose.model("User", userSchema);
```

Run project:

```
node index.js
```

Testing in Postman

Open Postman and test each API:

1. Create User (POST)

- URL: <http://localhost:3000/users>
- Body → Raw → JSON:

```
{
  "name": "vijay",
  "email": "vijay@gmail.com",
  "age": 22,
  "gender": "male"
}
```

2. Get All Users (GET)

- URL: <http://localhost:3000/users>

3. Get User by ID (GET)

- URL: http://localhost:3000/users/<user_id>

4. Update User (PUT)

- URL: http://localhost:3000/users/<user_id>
- Body:

```
{  
  "name": "vijay Updated",  
  "age": 23  
}
```

5. Delete User (DELETE)

- URL: http://localhost:3000/users/<user_id>

10. Create Data base by name college and a collection by name student_Details in MongoDB:

Step 1: Install MongoDB

- Download MongoDB shell from: <https://www.mongodb.com/try/download/shell>
- Install it with default settings.
- Ensure MongoDB Server (mongosh) is running.

Step 2: Open MongoDB Shell

- Open Command Prompt / Terminal.
- Type: **mongosh**
- This opens the MongoDB shell.

Step 3: Create Database

- In the shell, type: **use college**

Step 4: Create Collection

- To create a collection named student_Details, run:
db.createCollection("student_Details")

Step 5: Insert Sample Data

- Insert one student record:

```
db.student_details.insertOne(
{
  name: "Vijay",
  roll: "1013",
  age: 20,
  course: "ISE"
})
```

- Insert multiple records:

```
db.student_details.insertMany([
  { name: "kumar", roll: "1014", age: 21, course: "CSE" },
  { name: "vikram", roll: "1015", age: 22, course: "ECE" }
])
```

Step 6: View Data

- Show all documents in the collection: `db.student_Details.find().pretty()`

Step 7: Update the branch of the student_details from “ECE” to “MCE”.

- Cmd for update the branch:

```
db.student_details.updateMany(
  {branch:"ECE"},
  {$set:{branch:"MCE"}})
```

Step 8: Delete all the documents whose branch='CSE'.

- Following command to delete:

```
db.student_details.deleteOne({branch:"CSE"})
```

11. Demonstrate commands to create a docker file, build docker image with docker file, create docker container from docker image and run the docker container.

Step 1: Download Docker Desktop.

-
1. Go to Docker official website: <https://www.docker.com/products/docker-desktop>
 2. Download Docker Desktop for your windows
 3. Run the installer .exe → Click Next → Enable “*Use WSL 2*” → Finish.
 4. Verify Installation Open Command Prompt and type: **docker --version**

Step 2: Create a Dockerfile.

1. Open **Visual Studio Code**.
2. Create a new folder (example: hello-docker).
3. Inside the folder, create a file named **Dockerfile** (without extension).
4. Add the following content:

```
FROM python:3.9-slim  
WORKDIR /app  
COPY . /app  
CMD ["python", "-c", "print('Hello World')"]
```

Step 3: Build Docker Image.

1. Open VS Code terminal (or Command Prompt inside the project folder) and run:
docker build -t hello-world-image .
 - -t hello-world-image → Tags the image with the name hello-world-image.
 - . → Refers to the current directory where Dockerfile is located.
2. Check if the image is created successfully:

```
docker images
```

3. Create a container from the image:

```
docker create --name hello-world-container hello-world-image
```

- (--name hello-world-container)→ Assigns a name to the container.
- (hello-world-image) → Image from which container is created.

4. Run the container:

```
docker start hello-world-container
```

5. Check the output generated by the container:

```
docker logs hello-world-container
```

Commands to explore Docker image in VS code Output:

```
● PS C:\Users\vijay\Desktop\react project\hello-docker> docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
hello-world-image    latest         6e7ab81859e8   2 days ago     180MB
● PS C:\Users\vijay\Desktop\react project\hello-docker> docker create --name hello-world-container hello-world-image
f80e954797c8d9764d07ee31abadabd1f1ede97f1d4b7a67b8486f2c5f69fd17
● PS C:\Users\vijay\Desktop\react project\hello-docker> docker start hello-world-container
hello-world-container
● PS C:\Users\vijay\Desktop\react project\hello-docker> docker logs hello-world-container
Hello World
```

Run the Docker file in Dockerfile using Docker application (Docker):

